

I am using Mac, Linux, and Windows interchangeably with my development. Sometimes the file names mess up, some file names are supported in one platform, and others are not supported or the same file is not supporting another platform. What will be do? Currently, I have all problems with "5 Jun, 09.49.aac" & "5 يونيو.pdf". There are committed from Linux or Windows, I think it is from Windows. But I remember, I managed them on Linux. But my Mac have problems with this. What will do? Is git pre-commit can be help? I want to use a file names supported in all the platforms. I also want to keep a map of file names, which is changed for the gift friendly. What is your suggestion? Do you have any questions for me, clarify them first? I also think get pre commit, not cross-platform friendly.

Summary of the problem

You're switching files between **Linux, Windows, and macOS** and seeing filename problems: invisible or special characters, Unicode script issues, and platform-specific restrictions. The two examples you gave — `5 Jun, 09.49.aac` (contains an invisible/zero-width or non-standard space) and `5 يونيو.pdf` (Arabic script plus a space before the extension) — are classic cases caused by **Unicode normalization differences**, **forbidden/reserved characters**, and subtle invisible codepoints that some OSes tolerate and others don't.

Key causes to know

- **Unicode normalization (NFC vs NFD):** macOS historically uses a decomposed form (NFD) on HFS+, while Windows and many Linux filesystems use composed form (NFC). That can make the same logical filename appear different or fail to match across platforms.
- **Invisible / control characters and non-breaking spaces:** characters like zero-width joiner, zero-width space, non-breaking space, or other invisible codepoints break tools and shells on some platforms.
- **Forbidden/reserved characters and names (Windows):** `<:>"/\|?*`, trailing spaces or dots, and reserved names like `CON`, `PRN`, etc. are disallowed on Windows and cause errors when checking out or renaming files there.
- **Case sensitivity differences:** Windows and macOS (by default) are case-insensitive; Linux is case-sensitive. That can cause collisions or unexpected overwrites.
- **Git / macOS specifics:** Git has `core.precomposeunicode` to help with HFS+ decomposed names; without it you can get mismatches between what Git thinks and what the filesystem shows.

Sources: search results used to ground the above facts.

Practical, cross-platform strategy (recommended)

1. **Decide a canonical filename policy** (pick one and enforce it). Example policy:
 - Normalize Unicode to **NFC**.
 - Replace whitespace with `-` or `_` (no leading/trailing spaces).
 - Remove control/invisible characters.
 - Restrict to a safe character set: **ASCII letters, digits, -, _, .** (or allow Unicode but normalized).
 - Lowercase filenames (optional) to avoid case collisions.
2. **Enforce automatically with a pre-commit hook** (yes — pre-commit *can* help if you implement it correctly):
 - Use the [pre-commit framework] to run a hook that:
 - Normalizes Unicode to NFC.
 - Strips invisible/control characters.
 - Replaces forbidden characters with safe substitutes.
 - Fails the commit and prints the sanitized suggestion (or auto-renames and stages the rename).
 - This prevents new problematic names from entering the repo.
3. **Handle existing problematic files in the repo:**
 - Create a one-time migration script that:
 - Scans the repo for filenames with problematic codepoints or forbidden patterns.
 - Produces a **mapping file** (JSON/YAML) of `original -> sanitized` names.

- Uses `git mv` to rename files so history is preserved.

- Commit the mapping file into the repo so everyone can see the original→sanitized mapping.

4. Use Git config on macOS:

- Set `git config core.precomposeunicode true` on macOS machines to help Git handle decomposed filenames on HFS+/APFS correctly.

5. Document the policy and mapping:

- Add a `FILENAME_POLICY.md` describing allowed characters and the normalization rule.
- Keep `filename-map.json` (or similar) in the repo root with original and sanitized entries so tools and humans can translate names.

6. Optional: transliteration vs keep Unicode

- If you must support non-Latin scripts but want maximum cross-tool compatibility, either:
 - **Transliterate** to ASCII (e.g., `دیس گاہ` → `digital-quran.pdf`) and store transliteration in the map; or
 - **Keep Unicode** but enforce normalization to NFC and strict removal of invisible characters. Transliteration is safer for tooling; keeping Unicode is friendlier for users.

Concrete steps and example scripts

1) Quick checks to run locally

- Find filenames with non-ASCII or invisible chars:

bash

```
# show filenames with non-ASCII bytes
git ls-files -z | tr '\0' '\n' | awk '/[^\x00-\x7F]/'
# find files with control/invisible characters (example)
git ls-files -z | tr '\0' '\n' | grep -P '[\p{C}]'
```

2) Example Python sanitizer (can be used in a pre-commit hook)

python

```
# sanitize_filename.py
import sys, os, unicodedata, re

def sanitize(name):
    # Normalize to NFC
    name = unicodedata.normalize('NFC', name)
    # Remove control/invisible characters
    name = ''.join(ch for ch in name if unicodedata.category(ch)[0] != 'C')
    # Replace whitespace sequences with underscore
    name = re.sub(r'\s+', '_', name)
    # Replace forbidden Windows chars
    name = re.sub(r'[<>:"/\|?*]', '_', name)
    # Trim leading/trailing dots/underscores
    name = name.strip('._')
    return name

if __name__ == "__main__":
    for path in sys.argv[1:]:
        base = os.path.basename(path)
        new = sanitize(base)
        if new != base:
            print(f"{path} -> {os.path.join(os.path.dirname(path), new)}")
```

- Use this script in a pre-commit hook to **fail** the commit and show suggested renames, or to **auto-rename** and git add the new names.

3) Example pre-commit hook (shell)

- Add to `.git/hooks/pre-commit` or use the pre-commit framework:

bash

```
#!/bin/bash
# simple pre-commit: check staged filenames
staged=$(git diff --cached --name-only)
bad=0
for f in $staged; do
    new=$(python3 sanitize_filename.py "$f")
    if [ -n "$new" ] && [ "$new" != "$f" ]; then
        echo "Filename not allowed: $f -> $new"
    fi
done
```

```

    bad=1
  fi
done
[ $bad -eq 0 ]

```

4) One-time migration to rename existing files and create a map

- Script outline:
 - Walk `git ls-files`.
 - For each file compute `sanitized`.
 - If different, `git mv "orig" "sanitized"` (handle collisions).
 - Append mapping to `filename-map.json`.
 - Commit the renames and the map.

About your concerns: “pre-commit not cross-platform friendly”

- **Pre-commit hooks are cross-platform** if the hook scripts are written in a cross-platform language (Python, Node, or POSIX shell with care). Use the pre-commit framework (<https://pre-commit.com/>) which installs hooks in a reproducible way for all contributors. Make sure:
 - Hooks use Python 3 or a portable runtime.
 - You document `git config core.precomposeunicode true` for macOS users.
 - You include the mapping and migration steps in the repo so Windows/macOS/Linux users can run them consistently.

Suggested minimal immediate actions (priority list)

1. **Run a scanner** (scripts above) to find problematic filenames now.
2. **Set `git config core.precomposeunicode true` on macOS machines** (local config).
3. **Create and commit a filename policy doc** and a `filename-map.json` placeholder.
4. **Add a pre-commit hook** (or pre-commit framework) that normalizes filenames to NFC and rejects/renames unsafe names.
5. **Perform a one-time migration** to rename existing files with `git mv` and commit the mapping.

Questions for you (so I can tailor exact scripts)

1. **Do you prefer to keep Unicode filenames (e.g., Arabic) or transliterate them to ASCII?**
2. **Do you want automatic renaming (hook renames files for the user) or strict rejection (hook fails and asks user to rename manually)?**
3. **Are all contributors able to run Python 3 and install the pre-commit framework, or do you need a pure shell solution?**
4. **Do you use macOS with HFS+ or APFS, and can you set `git config core.precomposeunicode true` on those machines?**

Answer these and I'll produce:

- A ready-to-drop-in pre-commit hook (Python) that normalizes and optionally auto-renames, **plus**
- A one-time migration script that safely renames existing files with `git mv` and produces `filename-map.json`.

1. pros & cons of both? 2. strict rejection 3. pros & cons of python script, node script, pure shell, etc 4. i think yes

Short answer: Pick one canonical policy (I recommend NFC + safe-character fallback or transliteration), enforce it with a strict pre-commit hook that *rejects* unsafe names, and keep a committed `filename-map.json` for original→sanitized mapping. I'll give pros/cons, tool choices, and a step-by-step plan below.

Filename strategy comparison

Attribute	Keep Unicode (NFC)	Transliterate to ASCII
Compatibility	Good if normalized and sanitized; still can hit Windows reserved chars	Excellent across all tools and shells
Human readability	Best for native-language users	Loses original script but readable for global tooling
Tooling & search	Requires normalization handling; search may need Unicode-aware tools	Simple, predictable; search and scripts work reliably
Risk of invisible chars	Higher (must strip ZWJ, NBSP, etc.)	Low (transliteration removes them)
Mapping complexity	Map needed to show original → stored name	map still useful for user-facing label but simpler

Sources: Unicode normalization differences across OSes and why macOS uses decomposed forms.

1) Pros & cons of both (expanded)

• Keep Unicode (NFC)

- **Pros:** preserves original language and meaning; friendly to users who expect native script.
- **Cons:** must *normalize* to NFC; remove invisible/control characters, and avoid Windows forbidden characters; more fragile across tools and shells.

• Transliterate to ASCII

- **Pros:** maximum cross-platform reliability; simpler hooks and fewer surprises.
- **Cons:** may lose nuance; requires a consistent transliteration scheme and mapping file.

2) Enforcement: strict rejection (your choice)

- Implement a pre-commit hook that **validates staged filenames** and **fails the commit** if any file violates the policy. The hook should:
 - Normalize to **NFC** for comparison.
 - Detect and reject control/invisible characters, trailing dots/spaces, and Windows-forbidden characters.
 - Print a suggested sanitized filename and point to `FILENAME_POLICY.md` and `filename-map.json`.
- On macOS, set `git config core.precomposeunicode true` to reduce HFS+/Git mismatches.

3) Pros & cons of implementation languages/tools

• Python (recommended)

- **Pros:** excellent Unicode libraries (`unicodedata`), cross-platform, easy to write robust checks and mapping output (JSON).
- **Cons:** requires Python 3 on all contributors (usually available).

• Node (JavaScript)

- **Pros:** good if your team is JS-heavy; npm packages for transliteration and normalization.
- **Cons:** heavier runtime install for some contributors; Unicode handling less straightforward than Python.

• Pure shell (POSIX/bash)

- **Pros:** no extra runtime; simple checks possible.
- **Cons:** Unicode handling is fragile; different shells/locales cause inconsistencies—**not recommended** for strict Unicode normalization.

• pre-commit framework

- **Pros:** reproducible, installs hooks for all contributors; use Python hooks inside it.
- **Cons:** requires initial setup but pays off.

Actionable next steps (priority)

1. **Decide:** Unicode (NFC) or transliterate. (If unsure, choose transliterate for safety.)
2. Add `FILENAME_POLICY.md` and commit.

3. Create `sanitize` script in **Python** and a strict pre-commit hook (use pre-commit framework).
4. Run a one-time migration script that `git mv`'s problematic files and writes `filename-map.json`.
5. Instruct macOS users to set `git config core.precomposeunicode true`.

If you want, I'll **generate the Python sanitize script + a pre-commit config and a one-time migration script** tailored to *strict rejection* and produce the `FILENAME_POLICY.md` template next.

[Tower Git Client](#)

[Untracked files with unicode names | Tower Help \(Mac\)](#)

[Stack Overflow](#)

[Normalize filenames to NFC or not \(Unicode\) - Stack Overflow](#)

Transliterate to ASCII, but Map needed to show original → stored name "preserves original language and meaning; friendly to users who expect native script." → everyone is comfort with english is "git config core.precomposeunicode true" cause any problems in future? py script vs pre-commit faramework? I don't know nothing about pre-commit framework. explain that to me.

Short answer: Transliterate to ASCII for maximum cross-platform reliability and keep a committed `filename-map.json` that maps *original* → *stored* names. Set `core.precomposeunicode=true` on macOS; it's generally safe but not a silver bullet. Use a **Python sanitizer** run via the **pre-commit framework** to enforce strict rejection of unsafe names.

Summary of `core.precomposeunicode` and risks

What it does: On macOS, HFS/APFS stores filenames in a decomposed Unicode form (NFD); Linux/Windows typically use composed form (NFC). `git config core.precomposeunicode true` makes Git on macOS translate between the on-disk decomposed form and the repository's composed form so names match what Git expects. **Risks and compatibility:** Setting it on macOS is **commonly recommended** and usually safe for developer machines; it's a local setting (not enforced centrally) and helps avoid mismatches, but it doesn't remove invisible/control characters or Windows-forbidden names — you still need sanitization and a policy.

pre-commit framework explained

What it is: *pre-commit* is a small framework that manages Git hooks and installs them reproducibly for all contributors. Instead of each developer copying a hook script into `.git/hooks`, you add a `.pre-commit-config.yaml` and contributors run `pre-commit install`; the framework installs and runs hooks on every commit and can run them in CI too. **Why use it:** It standardizes hook installation, supports multi-language hooks, caches environments, and makes onboarding easier for teams.

Python script vs pre-commit framework

- **Python script alone**
 - **Pros:** Full Unicode control (`unicodedata`), easy JSON mapping, robust string handling, cross-platform.
 - **Cons:** You must ensure every contributor runs/install it (manual hook or instructions).
- **pre-commit framework + Python hook (recommended)**
 - **Pros:** Reproducible install, runs automatically on commit, easy to add other checks (linters, formatters), CI integration.
 - **Cons:** Small upfront setup (install `pre-commit`), but one-time and well documented.
- **Pure shell**
 - **Pros:** No extra runtime.
 - **Cons:** **Poor Unicode handling** and locale fragility — not suitable for strict normalization.

Practical recommendation and next steps

1. **Policy:** Transliterate to ASCII (e.g., `digital-quran.pdf`) and commit `filename-map.json` with original and stored entries. **Document** this in `FILENAME_POLICY.md`.
2. **macOS:** Instruct macOS devs to run `git config --global core.precomposeunicode true` (local setting) to reduce OS/HFS mismatches.

Git/FPS mismatches.

3. **Enforcement:** Create a **Python sanitizer** (normalize to NFC, strip control chars, transliterate) and add it as a **pre-commit hook** via the pre-commit framework so commits are **rejected** if filenames violate the policy.
4. **Migration:** Run a one-time script that scans `git ls-files`, computes sanitized names, `git mv`'s them, and writes `filename-map.json` before committing.

If you want, I'll generate the **Python sanitizer**, a `.pre-commit-config.yaml` example, and a one-time migration script next.

[Tower Git Client](#)

[Untracked files with unicode names | Tower Help \(Mac\)](#)

[Stack Overflow](#)

[Is it safe to set git's `core.precomposeunicode = true` globally?](#)

DEMO WATERMARK

choose pre-commit framework + Python hook i want to migrate existing files. is mac ok for that? or i need linux? or i need windows?

Short answer: You can migrate on any OS, but doing the rename/migration on a Linux machine (or any system that uses NFC) is the simplest and least error-prone; if you must use macOS, enable `core.precomposeunicode=true` and follow the careful steps below to avoid Unicode and Windows-name pitfalls.

Why OS choice matters

- **macOS stores filenames in a decomposed Unicode form (NFD)** while Linux and Windows typically use composed form (NFC). That difference can make the *same* logical name appear different on disk and confuse Git unless handled correctly.
- **Windows has reserved characters and names** (e.g. `<>:"/\\|?*;`, trailing spaces/dots, and reserved names like CON, PRN, COM1) that will break checkouts or file operations if your sanitized names collide with them.

Recommended migration approach (safe & reproducible)

1. Pick a migration host

- **Best:** Linux (NFC) — fewer surprises with composed filenames.
- **OK:** macOS — acceptable if you set `git config core.precomposeunicode true` before working and follow the steps below.
- **Avoid doing the initial bulk rename on Windows** because Windows will refuse forbidden names and may block some renames; use Windows only to verify afterwards.

2. Prepare setup and tooling

- Add `FILENAME_POLICY.md` describing transliteration → ASCII and the mapping file `filename-map.json`.
- Install the **pre-commit framework** and the Python sanitizer hook (we'll provide scripts). The framework ensures hooks run consistently for all contributors.

3. One-time scan and dry-run

- Run a scanner that lists all `git ls-files` entries that would change when transliterated/sanitized. Save the proposed mapping to `filename-map-dryrun.json`.
- Inspect for **collisions** (two originals mapping to the same ASCII name) and Windows reserved names; resolve collisions by adding suffixes or manual edits.

4. Perform safe renames with Git

- For each mapping `orig -> new` use `git mv --` to preserve history:

```
bash
git mv -- "orig" "new"
```
- Commit the renames and include `filename-map.json` in the same commit so the mapping is recorded.

5. Post-migration checks

- On macOS machines, ensure `git config core.precomposeunicode true` is set locally to avoid future display/checkout mismatches.
- On Windows, attempt a fresh clone to verify no forbidden names remain and that checkouts succeed.

6. Enforce going forward

- Add the Python sanitizer as a **pre-commit hook** that **rejects** commits with unsafe names and prints the suggested ASCII transliteration and mapping. Use the pre-commit framework so all contributors install the hook automatically.

Risks and gotchas

- **Collisions** when transliterating different originals to the same ASCII name — resolve before committing.
- **Invisible/control characters** may survive unless explicitly stripped; they can break shells and tools on some OSes.
- **macOS Git behavior:** without `core.precomposeunicode=true` you may see mismatches between what Git expects

macOS Git behavior without core.precomposeunicode=true you may see mismatches between what Git expects and what the filesystem shows.

- **Windows reserved names** will fail checkouts if not avoided; test a fresh clone on Windows before declaring migration complete.

Final note

Do the migration on Linux if possible. If you must use macOS, **set** `core.precomposeunicode=true` **first**, run the dry-run mapping, resolve collisions, then `git mv` and commit the mapping. I can now generate the **Python sanitizer**, a `.pre-commit-config.yaml` for the pre-commit framework, and a one-time migration script that produces `filename-map.json` and performs safe `git mv` operations.

[Tower Git Client](#)

[Untracked files with unicode names | Tower Help \(Mac\)](#)

help.interfaceware.com

[Windows Reserved File Names and Characters](#)

1. i choose linux - and waiting for its availability 2. ok 3. ok 4. ok 5. ok 6. ok Collisions when transliterating different originals to the same ASCII name - must handle Invisible/control characters may survive unless explicitly stripped - must handle macOS Git behavior - document it - but, don't rely on it Windows reserved names will fail checkouts if not avoided - must handle safe complete detailed plan, give all docs, give all scripts, etc i want commit everywhere, work everywhere

Bold summary: Transliterate filenames to ASCII, enforce a strict pre-commit Python hook (via the pre-commit framework) that rejects unsafe names, and run a one-time Linux migration that `git mv`'s files while producing `filename-map.json`. Follow the migration steps below, use the provided scripts, and test a fresh clone on Windows before declaring done.

Plan Overview

- **Goal:** commit and work on Linux, macOS, and Windows without filename errors.
- **Policy:** transliterate to ASCII, remove control/invisible characters, replace whitespace with `-`, forbid Windows reserved names, and lowercase filenames. **Documented and enforced** by pre-commit hooks.
- **Why:** macOS uses decomposed Unicode (NFD) while Linux/Windows use composed (NFC); Git has settings to help but you should not rely on macOS behavior alone.

Files and docs to add

- **FILENAME_POLICY.md** (short): rules, examples, transliteration policy, Windows reserved names list.
- **filename-map.json**: array of `{ "original": "...", "stored": "...", }` entries committed with the migration.
- **.pre-commit-config.yaml**: installs the Python hook so all contributors run the same checks.

Scripts (drop into repo)

sanitize_filename.py

python

```
#!/usr/bin/env python3
import sys, os, unicodedata, re
try:
    from unicode import unicode
except:
    unicode = None
```

```
WINDOWS_RESERVED = {"CON","PRN","AUX","NUL"} | {f"COM{i}" for i in range(10)} | {f"LPT{i}" for i in range(10)}
```

```
def translit(s):
    s = unicodedata.normalize('NFC', s)
    s = ''.join(ch for ch in s if unicodedata.category(ch)[0] != 'C')
    if unicode:
        s = unicode(s)
    else:
        s = ''.join(ch if ord(ch) < 128 else '_' for ch in s)
    s = re.sub(r'\s+', '-', s)
```

```

s = re.sub(r'[<>:"/\\|?*]', '_', s)
s = s.strip('._').lower()
if s.upper() in WINDOWS_RESERVED:
    s = s + '_file'
if not s:
    s = 'file'
return s

if __name__ == "__main__":
    for p in sys.argv[1:]:
        print(translit(os.path.basename(p)))

```

migrate_filenames.py (dry-run then apply)

bash

```

#!/bin/bash
# Usage: ./migrate_filenames.py --apply
python3 - <<'PY'
# (script: list git ls-files, compute new names via sanitize_filename.py,
# detect collisions, propose suffixes, print JSON mapping, if --apply then run git mv)
PY

```

(Full script will be provided on request; it performs dry-run, shows collisions, resolves by numeric suffix, and uses git mv --to preserve history.)

Pre-commit config

.pre-commit-config.yaml:

yaml

```

repos:
- repo: local
  hooks:
  - id: filename-sanitize
    name: filename-sanitize
    entry: python3 sanitize_commit_filenames.py
    language: system
    stages: [commit]

```

sanitize_commit_filenames.py should read staged names, run sanitize_filename.py, and **exit non-zero** if any staged file would change (printing suggested names and mapping).

Migration Steps (Linux recommended)

1. Clone repo on Linux (NFC).
2. Install Python3 and pre-commit.
3. Run dry-run migration script; inspect filename-map-dryrun.json.
4. Resolve collisions manually or accept numeric suffixes.
5. Run migration with --apply to git mv files and commit filename-map.json and FILENAME_POLICY.md.
6. Push and ask others to git pull and install pre-commit (pre-commit install).
7. Test fresh clone on Windows to ensure no forbidden names remain (Windows reserved names will break checkouts if present).

Collision and edge handling

- **Detect collisions** in dry run; append -1, -2 as needed and record in filename-map.json.
- **Strip invisible/control chars** explicitly in sanitizer (uses Unicode category checks).
- **Do not rely on macOS**; document git config core.precomposeunicode true as optional help but not required for correctness.

CI and verification

- Add a CI job that runs the sanitizer against git ls-files and fails the build if any violations exist. Use the same Python hook.

If you want, I'll now generate the full migrate_filenames.py implementation, the sanitize_commit_filenames.py hook, and a ready-to-commit FILENAME_POLICY.md and example filename-map.json.

[Tower Git Client](#)

[Untracked files with unicode names | Tower Help \(Mac\)](#)

pre-commit

pre-commit

migrate_filenames.py & sanitize_commit_filenames.py - give full scripts. also ready-to-commit
FILENAME_POLICY.md and example filename-map.json. Install Python3 and pre-commit - pre-commit how to? CI
and verification - need for codemagic.

Summary

You'll transliterate filenames to ASCII, enforce strict rejection, via a Python pre-commit hook (installed with the pre-commit framework), and run a one-time Linux migration that renames files with `git mv` and writes `filename-map.json`. Below are ready-to-play artifacts: two full scripts, the pre-commit hook and config, `FILENAME_POLICY.md`, an example `filename-map.json`, installation steps for Python and pre-commit, a safe migration procedure, and a Codemagic CI job to verify filenames.

Files to add to the repo

- `FILENAME_POLICY.md` — policy and examples.
- `sanitize_filename.py` — core transliteration + sanitization library.
- `sanitize_commit_filenames.py` — pre-commit hook that rejects unsafe staged filenames.
- `migrate_filenames.py` — one-time migration tool (dry run and apply).
- `.pre-commit-config.yaml` — pre-commit framework config.
- `filename-map.json` — produced by migration and committed.

sanitize_filename.py (full)

python

```
#!/usr/bin/env python3
"""
sanitize_filename.py
Library to transliterate and sanitize filenames to ASCII-safe form.
Uses unicode if available; falls back to replacing non-ASCII with '_'.
"""
import unicodedata, re, sys

try:
    from unicode import unicode
except Exception:
    unicode = None

WINDOWS_RESERVED = {"CON", "PRN", "AUX", "NUL"} | {f"COM{i}" for i in range(1,10)} | {f"LPT{i}" for i in range(1,10)}

def remove_controls(s: str) -> str:
    # Remove Unicode control and format characters (category C*)
    return ''.join(ch for ch in s if unicodedata.category(ch)[0] != 'C')

def normalize_unicode(s: str) -> str:
    # Normalize to NFC for consistent processing
    return unicodedata.normalize('NFC', s)

def transliterate_to_ascii(s: str) -> str:
    if unicode:
        return unicode(s)
    # fallback: replace non-ascii with underscore
    return ''.join(ch if ord(ch) < 128 else '_' for ch in s)

def sanitize_basename(name: str) -> str:
    name = normalize_unicode(name)
    name = remove_controls(name)
    name = transliterate_to_ascii(name)
    # Replace whitespace with single dash
    name = re.sub(r'\s+', '-', name)
    # Replace forbidden characters with underscore
    name = re.sub(r'[:<:"/\\|?*<]$', '_', name)
    # Collapse multiple underscores or dashes
    name = re.sub(r'[_-]{2,}', lambda m: m.group(0)[0], name)
    # Trim leading/trailing dots, spaces, underscores, dashes
    name = name.strip('._-')
    # Lowercase for consistency
    name = name.lower()
```

```

name = name.lower()
if not name:
    name = 'file'
# Avoid Windows reserved names
if name.upper() in WINDOWS_RESERVED:
    name = name + '_file'
return name

if __name__ == "__main__":
    # CLI: print sanitized basename for each arg
    for p in sys.argv[1:]:
        import os
        print(sanitize_basename(os.path.basename(p)))

```

sanitize_commit_filenames.py (full pre-commit hook)

python

```

#!/usr/bin/env python3
"""
sanitize_commit_filenames.py
Pre-commit hook: check staged filenames and reject commit if any would change.
Prints suggested sanitized names and a JSON mapping for manual review.
"""

import subprocess, json, sys, os
from sanitize_filename import sanitize_basename

def get_staged_files():
    # Get staged files (added/renamed/deleted)
    out = subprocess.check_output(['git', 'diff', '--cached', '--name-only', '--diff-filter=ACMR']).decode('utf-8')
    files = [line.strip() for line in out.splitlines() if line.strip()]
    return files

def main():
    staged = get_staged_files()
    if not staged:
        return 0
    mapping = []
    collisions = {}
    for f in staged:
        base = os.path.basename(f)
        newbase = sanitize_basename(base)
        if newbase != base:
            newpath = os.path.join(os.path.dirname(f), newbase)
            mapping.append({'original': f, 'suggested': newpath})
            collisions.setdefault(newpath, []).append(f)
    if not mapping:
        return 0
    # Detect collisions
    collision_found = False
    for newpath, originals in collisions.items():
        if len(originals) > 1:
            collision_found = True
    # Print helpful output
    print("\nERROR: Commit rejected due to filename policy violations.\n")
    print("Suggested renames:")
    for m in mapping:
        print(f"  {m['original']} -> {m['suggested']}")
    if collision_found:
        print("\nCollisions detected: multiple originals map to the same sanitized name.")
        print("Resolve collisions manually in filename-map.json or rename files before committing.\n")
    else:
        print("\nNo collisions detected. To apply renames locally, run the migration script or rename files and stage")
    # Write mapping to a temp file for convenience
    tmp = '.filename-suggested-map.json'
    with open(tmp, 'w', encoding='utf-8') as fh:
        json.dump(mapping, fh, indent=2, ensure_ascii=False)
    print(f"A suggested mapping was written to {tmp}. Add it to filename-map.json after review.\n")
    return 1

if __name__ == "__main__":
    rc = main()
    sys.exit(rc)

```

migrate_filenames.py (full)

python

```

#!/usr/bin/env python3
"""
migrate_filenames.py
Dry-run and apply migration of repository filenames to sanitized ASCII names.
Usage:
  python3 migrate_filenames.py --dry-run    # shows proposed mapping
  python3 migrate_filenames.py --apply      # performs git mv and writes filename-map.json
Notes:
  Run on Linux (recommended). Resolve collisions before applying if needed.
"""

import subprocess, json, os, sys
from sanitize_filename import sanitize_basename

```

```

def git_ls_files():
    out = subprocess.check_output(['git', 'ls-files', '-z']).decode('utf-8', errors='surrogateescape')
    files = [f for f in out.split('\0') if f]
    return files

def propose_mapping(files):
    mapping = []
    target_map = {}
    collisions = {}
    for f in files:
        base = os.path.basename(f)
        newbase = sanitize_basename(base)
        if newbase != base:
            newpath = os.path.join(os.path.dirname(f), newbase)
            # normalize './' prefix
            newpath = newpath.lstrip('./')
            mapping.append({'original': f, 'suggested': newpath})
            target_map.setdefault(newpath, []).append(f)
    for k, v in target_map.items():
        if len(v) > 1:
            collisions[k] = v
    return mapping, collisions

def apply_mapping(mapping):
    # Ensure no collisions
    targets = {}
    for m in mapping:
        orig = m['original']
        tgt = m['suggested']
        if tgt in targets:
            # collision: append numeric suffix
            i = 1
            base, ext = os.path.splitext(tgt)
            while True:
                candidate = "{base}-{i}{ext}"
                if candidate not in targets.values():
                    tgt = candidate
                    break
                i += 1
            m['suggested'] = tgt
        targets[orig] = tgt
    # Perform git mv operations
    for m in mapping:
        orig = m['original']
        tgt = m['suggested']
        os.makedirs(os.path.dirname(tgt) or '.', exist_ok=True)
        print(f"git mv -- \"{orig}\" \"{tgt}\"")
        subprocess.check_call(['git', 'mv', '--', orig, tgt])
    # Write filename-map.json
    with open('filename-map.json', 'w', encoding='utf-8') as fh:
        json.dump(mapping, fh, indent=2, ensure_ascii=False)
    print("\nMigration applied. Review filename-map.json, then commit the changes:")
    print("  git add filename-map.json")
    print("  git commit -m \"Migrate filenames to ASCII-safe names and add filename-map.json\"")

def main():
    apply = '--apply' in sys.argv
    files = git_ls_files()
    mapping, collisions = propose_mapping(files)
    if not mapping:
        print("No filenames need migration.")
        return 0
    print("Proposed mapping (first 200 entries):")
    for m in mapping[:200]:
        print(f"  {m['original']} -> {m['suggested']}")
    if collisions:
        print("\nCollisions detected for these targets:")
        for tgt, srcs in collisions.items():
            print(f"  {tgt}:")
            for s in srcs:
                print(f"    - {s}")
        print("\nResolve collisions manually before applying. You can edit the suggested names in filename-map.json a")
        if apply:
            print("Aborting apply due to collisions.")
            return 2
    if apply:
        apply_mapping(mapping)
    else:
        # write dry-run mapping
        with open('filename-map-dryrun.json', 'w', encoding='utf-8') as fh:
            json.dump(mapping, fh, indent=2, ensure_ascii=False)
        print("\nDry run mapping written to filename-map-dryrun.json.")
        print("If mapping looks good, run with --apply to perform git mv operations.")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

.pre-commit-config.yaml and hook install

```
.pre-commit-config.yaml
```

```
yaml
```

```
repos:
  - repo: local
    hooks:
      - id: filename-sanitize
        name: filename-sanitize
        entry: python3 sanitize_commit_filenames.py
        language: system
        stages: [commit]
```

How to install pre-commit for contributors

1. Ensure Python 3 is installed.
2. Install pre-commit once per machine:

```
bash
```

```
pip3 install --user pre-commit
```

3. From the repo root run:

```
bash
```

```
pre-commit install
```

This installs the hook into `.git/hooks` so `sanitize_commit_filenames.py` runs on every commit. To run hooks against all files once:

```
bash
```

```
pre-commit run --all-files
```

FILENAME POLICY.md (ready to commit)

```
markdown
```

```
# Filename Policy
```

Goal: Filenames stored in the repository must be ASCII-safe, predictable, and cross-platform friendly.

Rules

- Filenames are transliterated to ASCII and normalized.
- Whitespace is replaced with ``-``.
- Forbidden characters `<>:"/\"/>`

2. Ensure Python 3 and unicode are available:

bash

```
python3 -m pip install --user unicode pre-commit
```

3. Add the scripts and commit them (or pull them from a branch).
4. Run dry run:

bash

```
python3 migrate_filenames.py --dry-run
```

5. Inspect filename-map-dryrun.json and resolve collisions manually if any.
6. When satisfied, apply:

bash

```
python3 migrate_filenames.py --apply
```

7. Review changes, add filename-map.json and FILENAME_POLICY.md, then commit:

bash

```
git add filename-map.json FILENAME_POLICY.md
git commit -m "Migrate filenames to ASCII-safe names and add filename map and policy"
git push
```

8. Ask all contributors to pull and run:

bash

```
pip3 install --user pre-commit
pre-commit install
```

9. Verify on Windows by cloning the repo fresh on a Windows machine to ensure checkouts succeed.

Codemagic CI verification job

Add a Codemagic workflow step to fail builds if any filenames violate the policy. Example codemagic.yaml snippet:

yaml

```
workflows:
  check-filenames:
    name: Check Filenames
    max_build_duration: 60
    scripts:
      - name: Install Python deps
        script: |
          python3 -m pip install --user unicode
      - name: Run filename sanitizer check
        script: |
          python3 sanitize_commit_filenames.py || (echo "Filename policy violations detected. See output above." && e
```

Behavior: The hook script returns non-zero when violations exist, causing the Codemagic build to fail. Add this workflow to your CI triggers for PRs and main branch checks.

Final notes and tips

- **Collisions:** resolve in dry run before applying. The migration script will append numeric suffixes if collisions remain during apply, but manual resolution is cleaner.
- **Invisible characters:** the sanitizer explicitly removes Unicode control/format characters.
- **macOS:** document `git config --global core.precomposeunicode true` in FILENAME_POLICY.md but do not rely on it.
- **Backups:** create a branch and push before applying migration so you can revert if needed.
- **Testing:** after migration, test a fresh clone on Windows to confirm no checkout errors.

If you want, I will now:

- produce a single zip-ready patch with all files and executable permissions set, or
- generate a step-by-step shell command sequence to run the migration safely on your Linux machine.

Which would you like me to prepare next?

DEMO WATERMARK

```
try: from unidecode import unidecode except Exception: unidecode = None explain # Write mapping to a temp file
for convenience tmp = '.filename-suggested-map.json' with open(tmp, 'w', encoding='utf-8') as fh:
json.dump(mapping, fh, indent=2, ensure_ascii=False) print(f"A suggested mapping was written to {tmp}. Add it to
filename-map.json after review.\n") write to current folder is pre-commit available available as standalone
executable? without dependency on python (or with embedded python)? if it is embedded python - can it execute
hook py scripts too - so, i don't have to install python in my machine? is it available from homebrew/ macports (i
recommend brew) / apt (i commonly use ubuntu) / pacman / other linux package managers / winget / scoop /
choco /
```

Short answer: The try/except around unidecode makes transliteration *optional* — if Unidecode is installed it's used, otherwise the script falls back to a safe ASCII replacement. The temp JSON file is written to the *current working directory* (where you run the script). pre-commit is a Python-based tool (needs a Python runtime), but it's packaged for common package managers (Homebrew, apt, pip) so you usually don't have to install Python separately; there is no widely used single-file native binary that removes the Python dependency entirely.

1) What the try/except does (plain)

- try: from unidecode import unidecode attempts to import the optional **Unidecode** library.
- If import **succeeds**, unidecode() is available and used to produce readable ASCII transliterations (recommended).
- If import **fails**, the except sets unidecode = None so your code can detect that and fall back to a safe strategy (e.g., replace non-ASCII with a simple transliteration). This pattern makes the dependency optional and lets the script run even when Unidecode isn't installed; you get better transliteration when it is present.

2) Where the temp file is written

- tmp = '.filename-suggested-map.json' and open(tmp, ...) write to the **current working directory** — i.e., the directory where you run the Python process (commonly the repo root if you run the script there). If you want to *always* write to the repo root, compute an absolute path with os.path.join(os.getcwd(), tmp) OR git rev-parse --show-toplevel to find the repo root and write there.

Example (write to repo root):

python

```
import subprocess, os
repo_root = subprocess.check_output(['git', 'rev-parse', '--show-toplevel']).decode().strip()
tmp = os.path.join(repo_root, '.filename-suggested-map.json')
with open(tmp, 'w', encoding='utf-8') as fh:
    json.dump(mapping, fh, indent=2, ensure_ascii=False)
```

3) Is pre-commit a standalone executable?

- **No pure zero-Python binary:** pre-commit is a Python program and requires a Python runtime to run. The project manages hook environments and will create virtual environments for hooks as needed.
- **Packaged installers:** Many package managers provide pre-commit packages so installation is simple and the package manager will pull a Python dependency for you:
 - **Homebrew** formula exists (brew install pre-commit) and the formula depends on a Python package/runtime.
 - **Debian/Ubuntu** packages are available in distro repos (you can install pre-commit via apt).
 - **PyPI:** pip install pre-commit is the canonical install method. These installers mean you often don't need to manually install Python first — the package manager will satisfy that dependency for you.

4) Can pre-commit run Python hook scripts without Python installed locally?

- **pre-commit itself needs Python** to run. If you install pre-commit via a package manager, that installation typically provides or depends on Python so you won't have to separately pip install python.
- **Hook execution:** pre-commit creates isolated environments for hooks (Python hooks run in venvs it manages). You can also declare language: system hooks that run directly with the system interpreter, but that still requires the appropriate runtime to exist on the machine. See the pre-commit docs for details.

Quick install recommendations

- macOS (Homebrew): brew install pre-commit (brew will handle Python).
- Ubuntu: sudo apt install pre-commit OR pip3 install --user pre-commit (apt package exists).

- Cross-platform: `pip3 install --user pre-commit` then `pre-commit install` in repo root.

If you want, I'll: (A) add the repo-root write snippet into your hook scripts, and (B) produce exact `brew / apt / pip` commands and a short checklist for onboarding contributors on macOS, Linux, and Windows.

[pre-commit](#)

[pre-commit](#)

[Homebrew Formulae](#)

[pre-commit — Homebrew Formulae](#)

transliteration optional - transliteration is must. transliteration vs safe ASCII replacement? write to repo root - give full script. produce exact `brew / apt / pip` commands and a short checklist for onboarding contributors on macOS, Linux, and Windows - update docs too

Short answer: Make transliteration mandatory (use Unicode) and write all mapping files to the repository root. Use the Python sanitizer + pre-commit from `work`; install pre-commit via Homebrew or pip and require Unicode as a dependency. Below are the pros/cons, exact scripts (writing to repo root), install commands, and a short onboarding checklist.

Transliteration vs safe ASCII replacement

- **Transliteration (must):** *Pros:* preserves readable, pronounceable ASCII equivalents (e.g., `دِجِیٲل` → `digital`); better UX and searchability. *Cons:* requires Unicode dependency and may imperfectly map some scripts.
- **Safe ASCII replacement:** *Pros:* zero external dependency; deterministic. *Cons:* produces unreadable names (`_` or `hex`), worse UX, higher collision risk. **Decision:** transliteration is required; fallback behavior is not acceptable.

Scripts (write mapping to repo root)

sanitize_filename.py

```
python

#!/usr/bin/env python3
import unicodedata, re, os, sys
try:
    from unicode import unicode
except Exception:
    print("ERROR: 'Unicode' is required. Install with: pip3 install --user Unicode", file=sys.stderr)
    sys.exit(2)

WINDOWS_RESERVED = {"CON", "PRN", "AUX", "NUL"} | {"COM{i}" for i in range(1,10)} | {"LPT{i}" for i in range(1,10)}

def remove_controls(s):
    return ''.join(ch for ch in s if unicodedata.category(ch)[0] != 'C')

def sanitize_basename(name):
    name = unicodedata.normalize('NFC', name)
    name = remove_controls(name)
    name = unicode(name)
    name = re.sub(r'\s+', '-', name)
    name = re.sub(r'[\<:"/\\|?*\]', '_', name)
    name = re.sub(r'[_-]{2,}', lambda m: m.group(0)[0], name)
    name = name.strip('._-').lower()
    if not name:
        name = 'file'
    if name.upper() in WINDOWS_RESERVED:
        name = name + '_file'
    return name

if __name__ == "__main__":
    for p in sys.argv[1:]:
        print(sanitize_basename(os.path.basename(p)))
```

sanitize_commit_filenames.py (pre-commit hook; writes mapping to repo root)

```
python

#!/usr/bin/env python3
import subprocess, json, sys, os
from sanitize_filename import sanitize_basename

def repo_root():
    return subprocess.check_output(['git', 'rev-parse', '--show-top-level']).decode().strip()
```

```

    return subprocess.check_output(['git', 'rev-parse', '--show-toplevel']).decode().strip()

def get_staged_files():
    out = subprocess.check_output(['git', 'diff', '--cached', '--name-only', '--diff-filter=ACMR']).decode()
    return [l for l in out.splitlines() if l]

def main():
    root = repo_root()
    staged = get_staged_files()
    mapping = []
    for f in staged:
        newbase = sanitize_basename(os.path.basename(f))
        if newbase != os.path.basename(f):
            newpath = os.path.join(os.path.dirname(f), newbase).lstrip('/')
            mapping.append({'original': f, 'suggested': newpath})
    if not mapping:
        return 0
    tmp = os.path.join(root, '.filename-suggested-map.json')
    with open(tmp, 'w', encoding='utf-8') as fh:
        json.dump(mapping, fh, indent=2, ensure_ascii=False)
    print("Commit rejected: filename policy violation. Suggested mapping written to", tmp)
    return 1

if __name__ == "__main__":
    sys.exit(main())

```

migrate_filenames.py (previously provided) creates `filename-map.json` to repo root; run it on Linux.

Exact install commands

- **macOS (Homebrew):**

bash

```
brew install pre-commit
python3 -m pip install --user Unidecode
```

(Homebrew provides a pre-commit formula).

- **Ubuntu / Debian (recommended via pip):**

bash

```
python3 -m pip install --user pre-commit Unidecode
```

(pre-commit is also on PyPI).

- **Cross-platform (pip):**

bash

```
python3 -m pip install --user pre-commit Unidecode
```

- **Windows:** install Python (from python.org or winget), then:

powershell

```
python -m pip install --user pre-commit Unidecode
```

Official docs and installation guidance are on the pre-commit site.

(Using pip is the most portable; Homebrew provides a packaged formula on macOS).

Onboarding checklist (add to docs)

1. Install Python 3.10+ (or use system Python).
2. Install tools:
 - macOS: `brew install pre-commit` then `python3 -m pip install --user Unidecode`.
 - Linux/Windows: `python3 -m pip install --user pre-commit Unidecode`.
3. From repo root run: `pre-commit install`.
4. Run `pre-commit run --all-files` to validate.
5. If migration was applied, pull latest and verify `filename-map.json`.
6. Do not bypass hooks; commits with filename violations will be rejected.

If you want, I'll (A) update `FILENAME_POLICY.md` to require Unidecode and include these install commands, and (B) produce a short Codemagic step that runs `pre-commit run --all-files` in CI. Which do you want next?

[Homebrew Formulae](#)

can mac use pip? also document python install too. i want A & B

Overview

You can use pip on macOS. I updated the artifacts to **require** Unidecode, write mapping files to the **repository root**, and added exact install commands plus an onboarding checklist for macOS, Linux, and Windows. I also included a Codemagic CI snippet that runs the same checks.

Below you'll find:

- **Updated scripts** (full, ready to drop into repo) that write mapping files to the repo root and require Unidecode.
- **Updated** FILENAME_POLICY.md (requires Unidecode, install steps).
- **Example** filename-map.json
- **Exact install commands** for macOS (Homebrew + pip), Ubuntu, generic pip, and Windows.
- **Onboarding checklist** for contributors.
- **Codemagic CI** snippet to verify filenames.

Scripts (write mapping to repo root; Unidecode required)

sanitize_filename.py

python

```
#!/usr/bin/env python3
"""
sanitize_filename.py
Transliterate and sanitize filenames to ASCII-safe form.
Requires: Unidecode (pip install Unidecode)
"""
import unicodedata, re, os, sys
try:
    from unidecode import unidecode
except Exception:
    print("ERROR: 'Unidecode' is required. Install with: pip3 install --user Unidecode", file=sys.stderr)
    sys.exit(2)

WINDOWS_RESERVED = {"CON", "PRN", "AUX", "NUL"} | {f"COM{i}" for i in range(1,10)} | {f"LPT{i}" for i in range(1,10)}

def remove_controls(s):
    return ''.join(ch for ch in s if unicodedata.category(ch)[0] != 'C')

def sanitize_basename(name):
    name = unicodedata.normalize('NFC', name)
    name = remove_controls(name)
    name = unidecode(name)
    name = re.sub(r'\s+', '-', name)
    name = re.sub(r'[<:;"/\\|?*\]', '_', name)
    name = re.sub(r'[_]{2,}', lambda m: m.group(0)[0], name)
    name = name.strip('._-').lower()
    if not name:
        name = 'file'
    if name.upper() in WINDOWS_RESERVED:
        name = name + '_file'
    return name

if __name__ == "__main__":
    for p in sys.argv[1:]:
        print(sanitize_basename(os.path.basename(p)))
```

sanitize_commit_filenames.py (pre-commit hook; writes suggested mapping to repo root and rejects commit)

python

```
#!/usr/bin/env python3
"""
```

```

sanitize_commit_filenames.py
Pre-commit hook: check staged filenames and reject commit if any would change.
Writes suggested mapping to repo root as .filename-suggested-map.json
Requires: Unidecode (pip install Unidecode)
"""
import subprocess, json, sys, os
from sanitize_filename import sanitize_basename

def repo_root():
    return subprocess.check_output(['git', 'rev-parse', '--show-toplevel']).decode().strip()

def get_staged_files():
    out = subprocess.check_output(['git', 'diff', '--cached', '--name-only', '--diff-filter=ACMR']).decode()
    return [l for l in out.splitlines() if l]

def main():
    root = repo_root()
    staged = get_staged_files()
    mapping = []
    for f in staged:
        newbase = sanitize_basename(os.path.basename(f))
        if newbase != os.path.basename(f):
            newpath = os.path.join(os.path.dirname(f), newbase).lstrip('./')
            mapping.append({'original': f, 'suggested': newpath})
    if not mapping:
        return 0
    tmp = os.path.join(root, '.filename-suggested-map.json')
    with open(tmp, 'w', encoding='utf-8') as fh:
        json.dump(mapping, fh, indent=2, ensure_ascii=False)
    print("\nCommit rejected: filename policy violations detected.")
    print("A suggested mapping was written to:", tmp)
    print("Please review, rename files locally (or run the migration), stage changes, and commit again.\n")
    return 1

if __name__ == "__main__":
    sys.exit(main())

```

migrate_filenames.py (dry-run and apply; writes filename-map.json to repo root)

python

```

#!/usr/bin/env python3
"""
migrate_filenames.py
Dry-run and apply migration of repository filenames to sanitized ASCII names.
Usage:
    python3 migrate_filenames.py --dry-run
    python3 migrate_filenames.py --apply
Run on Linux (recommended). Requires Unidecode.
"""
import subprocess, json, os, sys
from sanitize_filename import sanitize_basename

def git_ls_files():
    out = subprocess.check_output(['git', 'ls-files', '-z']).decode('utf-8', errors='surrogateescape')
    files = [f for f in out.split('\0') if f]
    return files

def propose_mapping(files):
    mapping = []
    target_map = {}
    collisions = {}
    for f in files:
        base = os.path.basename(f)
        newbase = sanitize_basename(base)
        if newbase != base:
            newpath = os.path.join(os.path.dirname(f), newbase).lstrip('./')
            mapping.append({'original': f, 'suggested': newpath})
            target_map.setdefault(newpath, []).append(f)
    for k, v in target_map.items():
        if len(v) > 1:
            collisions[k] = v
    return mapping, collisions

def apply_mapping(mapping, repo_root):
    # Resolve collisions by numeric suffix if needed
    used_targets = set()
    for m in mapping:
        tgt = m['suggested']
        if tgt in used_targets:
            base, ext = os.path.splitext(tgt)
            i = 1
            while True:
                candidate = f"{base}-{i}{ext}"
                if candidate not in used_targets:
                    tgt = candidate
                    break
            i += 1
        m['suggested'] = tgt
        used_targets.add(m['suggested'])
    # Perform git mv operations

```

```

for m in mapping:
    orig = m['original']
    tgt = m['suggested']
    os.makedirs(os.path.dirname(tgt) or '.', exist_ok=True)
    print(f"git mv -- \"{orig}\" \"{tgt}\"")
    subprocess.check_call(['git', 'mv', '--', orig, tgt])
# Write filename-map.json to repo root
outpath = os.path.join(repo_root, 'filename-map.json')
with open(outpath, 'w', encoding='utf-8') as fh:
    json.dump(mapping, fh, indent=2, ensure_ascii=False)
print("\nMigration applied. filename-map.json written to:", outpath)
print("Review changes, then run: git add filename-map.json && git commit -m \"Migrate filenames to ASCII-safe names\"")

def main():
    apply = '--apply' in sys.argv
    files = git_ls_files()
    mapping, collisions = propose_mapping(files)
    repo_root = subprocess.check_output(['git', 'rev-parse', '--show-toplevel']).decode().strip()
    if not mapping:
        print("No filenames need migration.")
        return 0
    print("Proposed mapping (first 200 entries):")
    for m in mapping[:200]:
        print(f"  {m['original']} -> {m['suggested']}")
    if collisions:
        print("\nCollisions detected for these targets:")
        for tgt, srcs in collisions.items():
            print(f"  {tgt}:")
            for s in srcs:
                print(f"    {s}")
        print("\nResolve collisions manually before applying or edit filename-map-dryrun.json.")
        if apply:
            print("Aborting apply due to collisions.")
            return 2
    if apply:
        apply_mapping(mapping, repo_root)
    else:
        dry_path = os.path.join(repo_root, 'filename-map-dryrun.json')
        with open(dry_path, 'w', encoding='utf-8') as fh:
            json.dump(mapping, fh, indent=2, ensure_ascii=False)
        print("\nDry run mapping written to:", dry_path)
        print("If mapping looks good, run with --apply to perform git mv operations.")
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

.pre-commit-config.yaml

yaml

```

repos:
- repo: local
  hooks:
  - id: filename-sanitize
    name: filename-sanitize
    entry: python3 sanitize_commit_filenames.py
    language: system
    stages: [commit]

```

FILENAME_POLICY.md (updated; ready to commit)

markdown

Filename Policy

Goal: Filenames stored in the repository must be ASCII-safe, predictable, and cross-platform friendly.

Requirements

- Transliteration is mandatory. All filenames are transliterated to ASCII using `Unicode`.
- Filenames are normalized to NFC before transliteration.
- Whitespace is replaced with `_`.
- Forbidden characters `<>:"/|?*` are replaced with `_`.
- Control and invisible characters are removed.
- Filenames are lowercased.
- Trailing dots and spaces are removed.
- Windows reserved names (CON, PRN, AUX, NUL, COM1..COM9, LPT1..LPT9) are avoided; if encountered, `_file` is appended.
- If transliteration causes collisions, numeric suffixes `-1`, `-2` are appended.

Process

- New commits are validated by a pre-commit hook and will be rejected if filenames violate the policy.
- A one-time migration script `migrate_filenames.py` is used to rename existing files and produce `filename-map.json`.
- `filename-map.json` maps original filenames to stored filenames for user reference.

Tools required

- Python 3.10+ (or 3.8+)
- `Unicode` Python package
- `pre-commit` (for hook installation)

Install commands (examples)

pre-commit commands (examples):

- macOS (Homebrew + pip):
 - `brew install pre-commit`
 - `python3 -m pip install --user Unidecode`
- Ubuntu / Debian:
 - `python3 -m pip install --user pre-commit Unidecode`
- Windows (PowerShell):
 - `python -m pip install --user pre-commit Unidecode`

****Notes****

- macOS users may set `git config --global core.precomposeunicode true` to reduce Git/HFS filename mismatches, but do
- Run `pre-commit install` after installing `pre-commit`.

Example filename-map.json

json

```
[
  {
    "original": "ٱٱٱٱٱpdf",
    "suggested": "digital-quran.pdf"
  },
  {
    "original": "5 Jun, 09.49.aac",
    "suggested": "5-jun-09-49.aac"
  }
]
```

Exact install commands and Python install documentation

macOS (recommended)

1. **Install Homebrew** (if not installed):

bash

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. **Install pre-commit via Homebrew:**

bash

```
brew install pre-commit
```

3. **Install Unidecode** (and ensure python3 is available):

bash

```
python3 -m pip install --user Unidecode
```

4. **Install hooks in repo:**

bash

```
pre-commit install
pre-commit run --all-files
```

Can macOS use pip? Yes. macOS ships with or can install Python 3; pip is available as pip3 or via python3 -m pip. Homebrew's python or the official Python installer are common ways to get Python.

Ubuntu / Debian (example)

bash

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
python3 -m pip install --user pre-commit Unidecode
pre-commit install
pre-commit run --all-files
```

Generic pip (cross-platform)

bash

```
python3 -m pip install --user pre-commit Unidecode
pre-commit install
pre-commit run --all-files
```

Windows (PowerShell)

1. Install Python from python.org or via winget:

powershell

```
winget install Python.Python.3
```

2. Install packages:

powershell

```
python -m pip install --user pre-commit Unidecode
pre-commit install
pre-commit run --all-files
```

Onboarding checklist (add to repo README or docs)

1. Install Python 3 (3.8+; 3.10+ recommended).

2. Install pre-commit and Unidecode:

- macOS: brew install pre-commit then python3 -m pip install --user Unidecode
- Linux/Windows: python3 -m pip install --user pre-commit Unidecode

3. From repo root run:

bash

```
pre-commit install
pre-commit run --all-files
```

4. If you see filename violations, review `.filename-suggested-map.json` in repo root, rename files locally (or run the migration on Linux), stage changes, and commit again.

5. Do not bypass hooks. If you must, discuss with the team and follow the migration process.

Codemagic CI snippet (verify filenames)

Add this to your Codemagic workflow to fail builds when filename policy violations exist:

yaml

```
workflow:
  check_filenames:
    name: Check Filenames
    scripts:
      - name: Install Python deps
        script: |
          python3 -m pip install --user Unidecode pre-commit
      - name: Run filename sanitizer check
        script: |
          # Run the pre-commit hook script directly to check repository files
          python3 sanitize_commit_filenames.py || (echo "Filename policy violations detected. See .filename-suggested
```

Behavior: the script returns non-zero when violations exist, causing the build to fail.

Final notes and recommended next actions

- **Run migration on Linux** when available: clone repo on Linux, install Unidecode, run `python3 migrate_filenames.py --dry-run`, inspect `filename-map-dryrun.json`, resolve collisions, then `--apply`. Commit `filename-map.json` and `FILENAME_POLICY.md`.
- **Ask all contributors** to install pre-commit and Unidecode and run `pre-commit install`.
- **Test a fresh clone on Windows** after migration to ensure checkouts succeed.

If you want, I'll:

- produce a single patch (diff) with all files added and executable bits set, or
- generate a short shell command sequence you can paste on your Linux machine to run the migration safely (dry-run then apply).

on macos, go with brew python

Short summary

Use **Homebrew Python** on macOS. Install python3 via Homebrew, then install pre-commit and Unidecode with `python3 -m pip`. I'll give exact commands, PATH notes, an updated onboarding checklist, and an updated FILENAME_POLICY.md that reflects the macOS brew-Python recommendation.

Why use Homebrew Python on macOS

- Homebrew Python is up-to-date, isolated from the system Python, and integrates well with developer workflows.
- `python3 -m pip` from Homebrew installs packages into a user environment that's easy to manage.
- This avoids relying on the macOS-supplied Python (which may be older or absent) and makes behavior consistent with Linux CI and developer machines.

Exact commands for macOS (Homebrew Python workflow)

1. Install Homebrew (if you don't have it):

bash

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Install Python via Homebrew:

bash

```
brew update  
brew install python
```

This installs python3 and pip3 into Homebrew's prefix (e.g., `/usr/local` or `/opt/homebrew`).

3. Ensure Homebrew Python is first on PATH (example for zsh; adjust for bash/fish):

bash

```
# For Apple Silicon (M1/M2) Homebrew default  
echo 'eval "$(/opt/homebrew/bin/brew shellenv)'" >> ~/.zprofile  
eval "$(/opt/homebrew/bin/brew shellenv)"
```

```
# For Intel Homebrew default  
echo 'eval "$(/usr/local/bin/brew shellenv)'" >> ~/.zprofile  
eval "$(/usr/local/bin/brew shellenv)"
```

Confirm:

bash

```
which python3  
python3 --version  
which pip3  
pip3 --version
```

4. Install required Python packages (user scope):

bash

```
python3 -m pip install --upgrade --user pre-commit Unidecode
```

This installs pre-commit and Unidecode into your user site packages. pre-commit will be available as a CLI if your user bin is on PATH.

5. Add pip user bin to PATH if needed (common location):

bash

```
# macOS typical user bin  
echo 'export PATH="$HOME/Library/Python/3.*/bin:$PATH"' >> ~/.zprofile  
# Replace 3.* with your minor version if you want (e.g., 3.11)  
source ~/.zprofile
```

Confirm pre-commit is available:

bash

```
pre-commit --version
```

6. Install the repo hooks (run once per repo):

bash

```
cd /path/to/your/repo
pre-commit install
pre-commit run --all-files
```

Notes about alternatives

- You can also brew install pre-commit (Homebrew formula) instead of pip if you prefer a packaged install; still install Unidecode via pip.
- Using `python3 -m pip install --user ...` is portable and consistent across macOS and Linux.

Updated onboarding checklist (macOS with Homebrew, Python, Linux, Windows)

Before you start (all platforms)

- Have Git installed and configured.
- Use the repository's `FILENAME_POLICY.md` and follow the migration instructions if present.

macOS (Homebrew Python)

1. Install Homebrew if needed.
2. `brew install python`
3. `python3 -m pip install --upgrade --user pre-commit Unidecode`
4. Ensure user pip bin is in PATH (see commands above).
5. From repo root:

```
bash
pre-commit install
pre-commit run --all-files
```

6. If pre-commit reports filename violations, review `.filename-suggested-map.json` in repo root, rename files locally (or coordinate migration), stage, and commit.

Linux (recommended for migration)

1. Install Python 3 and pip (example Ubuntu):

```
bash

sudo apt update
sudo apt install -y python3 python3-venv python3-pip
```

2. `python3 -m pip install --user pre-commit Unidecode`
3. `pre-commit install` and `pre-commit run --all-files` in repo root.

Windows

1. Install Python 3 (from python.org or winget).
2. `python -m pip install --user pre-commit Unidecode`
3. Add the user Scripts folder to PATH if needed (e.g., `%USERPROFILE%\AppData\Roaming\Python\Python3x\Scripts`).
4. `pre-commit install` and `pre-commit run --all-files`.

Updated `FILENAME_POLICY.md` (ready to commit)

markdown

Filename Policy

Goal: Filenames stored in the repository must be ASCII-safe, predictable, and cross-platform friendly.

Requirements

- **Transliteration is mandatory.** All filenames are transliterated to ASCII using ``Unidecode``.
- Filenames are normalized to NFC before transliteration.
- Whitespace is replaced with ``_``.
- Forbidden characters ``<>:"/\\|?*'`` are replaced with ``_``.
- Control and invisible characters are removed.
- Filenames are lowercased.

- trailing dots and spaces are removed.
- Windows reserved names (CON, PRN, AUX, NUL, COM1..COM9, LPT1..LPT9) are avoided; if encountered, `\_file` is appended.
- If transliteration causes collisions, numeric suffixes `\_1`, `\_2` are appended.

Tools required

- Python 3.8+ (3.10+ recommended)
- `Unidecode` Python package (mandatory)
- `pre-commit` (for hook installation)

macOS (recommended setup)

- Use Homebrew Python:
 - `brew install python`
 - `python3 -m pip install --upgrade --user pre-commit Unidecode`
- Ensure your user pip bin is on PATH (e.g., `~/zprofile` updated).
- From repo root: `pre-commit install` and `pre-commit run --all-files`

Process

- New commits are validated by a pre-commit hook and will be rejected if filenames violate the policy.
- A one-time migration script `migrate\_filenames.py` is used to rename existing files and produce `filename-map.json`
- `filename-map.json` maps original filenames to stored filenames for user reference.

Notes

- macOS users may set `git config --global core.precommitunicode true` to reduce Git/HFS filename mismatches, but do
- Migration should be performed on Linux when possible; macOS is acceptable if Homebrew Python is used and you follow

Codemagic CI note

Use the same Python environment in CI. Example step (Codemagic) to install Unidecode and run the check:

yaml

- ```
- name: Install Python deps
 script: |
 python3 -m pip install --user Unidecode pre-commit
- name: Run filename sanitizer check
 script: |
 python3 sanitize_commit_filenames.py || (echo "Filename policy violations detected. See .filename-suggested-map.j
```

### Next steps I can prepare for you

- Create a single patch (diff) that adds the scripts, .pre-commit-config.yaml, FILENAME\_POLICY.md, and example filename-map.json to the repo.
- Produce the exact shell command sequence you can paste on your Linux machine to run the migration (dry-run then apply).

Tell me which of those you want and I'll produce it.

on mac, use brew pre-commit. brew packages always will be on path. on windows - use winget python. give complete ci job

### Overview

Below is a **complete Codemagic CI workflow** that verifies filename policy across macOS, Linux, and Windows runners. Each job installs the required tooling, ensures Unidecode is available, runs the repository's Python filename checker (sanitize\_commit\_filenames.py) and the pre-commit hooks, and fails the build if any filename violations are detected. The macOS job uses **Homebrew** to install pre-commit as you requested; the Windows job uses **winget** to install Python.

### What this workflow does (high level)

- Installs Python and required packages (pre-commit, Unidecode) on each runner.
- Ensures pre-commit is available (macOS via Homebrew; Linux/Windows via pip).
- Runs the repository filename check script sanitize\_commit\_filenames.py. If it returns non-zero, the job fails and prints .filename-suggested-map.json path.
- Runs pre-commit run --all-files to run any other hooks you may have.
- Exits non-zero on any violations so PRs and merges are blocked until filenames comply.

### Complete codemagic.yaml (drop into your Codemagic project)

yaml

#### WORKFLOWS:

```
check-filenames-matrix:
 name: Check Filenames (macOS, Linux, Windows)
 max_build_duration: 60
 environment:
 vars:
 # Optional: set PYTHONUSERBASE to a known location if you want to control user installs
 PYTHONUSERBASE: $HOME/.local
 scripts:
 - name: Prepare workspace
 script: |
 set -e
 echo "Running filename checks on runner: $CM_BUILD_PLATFORM"
 cd "$CM_BUILD_DIR"
 - name: macOS: Install tooling (macOS runner)
 script: |
 if ["$CM_BUILD_PLATFORM" = "macos"]; then
 # Ensure Homebrew is available on macOS runners (Codemagic macOS images include brew)
 echo "macOS runner detected. Installing pre-commit via brew and Unidecode via pip"
 brew update || true
 brew install pre-commit || true
 python3 -m pip install --upgrade --user Unidecode
 # Ensure user pip bin is on PATH for pre-commit if needed
 export PATH="$HOME/Library/Python/3.x/bin:$PATH" || true
 pre-commit --version || true
 fi
 - name: Linux: Install tooling (Linux runner)
 script: |
 if ["$CM_BUILD_PLATFORM" = "linux"]; then
 echo "Linux runner detected. Installing Python packages via pip"
 python3 -m pip install --user --upgrade pip
 python3 -m pip install --user pre-commit Unidecode
 export PATH="$HOME/.local/bin:$PATH"
 pre-commit --version || true
 fi
 - name: Windows: Install tooling (Windows runner)
 script: |
 if ["$CM_BUILD_PLATFORM" = "windows"]; then
 echo "Windows runner detected. Installing Python via winget and Python packages"
 # Install Python via winget (may require elevated privileges on some runners)
 winget install --id Python.Python.3 -e --silent || true
 # Ensure python is on PATH; use python -m pip
 python -m pip install --user --upgrade pip
 python -m pip install --user pre-commit Unidecode
 # Add user scripts to PATH for this session
 export PATH="%APPDATA%\Python\Scripts;%PATH%" || true
 pre-commit --version || true
 fi
 - name: Run filename sanitizer script
 script: |
 set -e
 cd "$CM_BUILD_DIR"
 echo "Running sanitize_commit_filenames.py to check staged/committed filenames"
 # Run the checker; it writes .filename-suggested-map.json to repo root on violations
 python3 sanitize_commit_filenames.py || (echo "Filename policy violations detected. See .filename-suggested-map.json" && exit 1)
 - name: Run pre-commit hooks (all files)
 script: |
 set -e
 cd "$CM_BUILD_DIR"
 # Run pre-commit against all files to ensure no other hook fails
 pre-commit run --all-files || (echo "pre-commit hooks failed. Fix issues and push again." && exit 1)
Run this workflow on all supported platforms
triggers:
 - push
 - pull_request
Platform matrix: Codemagic will run the same workflow on each platform if configured in project settings.
```

## Notes and runner specifics

### • macOS runner

- Uses brew install pre-commit. Homebrew packages are on PATH by default on Codemagic macOS images.
- Installs Unidecode via python3 -m pip install --user Unidecode.

### • Linux runner

- Uses pip to install pre-commit and Unidecode into the user site (\$HOME/.local/bin added to PATH).

### • Windows runner

- Uses winget to install Python, then python -m pip install --user pre-commit Unidecode.
- Some Windows runners may already have Python; winget step is safe to skip if Python exists.
- Ensure the user Scripts folder is on PATH for the session so pre-commit is callable.

- **Codemagic variables**

- \$CM\_BUILD\_DIR is the repo root on the runner; scripts `cd` into it before running checks.
- \$CM\_BUILD\_PLATFORM is used to branch platform-specific install steps.

## Failure behavior and developer feedback

- If `sanitize_commit_filenames.py` finds violations it:
  - Exits non-zero.
  - Writes `.filename-suggested-map.json` to the repository root (the job prints its path).
  - The CI job fails and the PR shows the failure; developers should inspect the suggested map, rename files locally (or run the migration on Linux), stage, and push again.
- `pre-commit run --all-files` ensures other hooks (commit linters, linters) also pass in CI.

## Add to repo and CI integration checklist

1. Add the scripts to repo root: `sanitize_filenames.py`, `sanitize_commit_filenames.py`, `migrate_filenames.py`, `.pre-commit-config.yaml`, `FILENAME_POLICY.md`.
2. Commit and push these files to a branch.
3. Add `codemagic.yaml` to the project (root) with the workflow above.
4. Trigger a PR — Codemagic will run the workflow on configured platforms.
5. If failures occur, inspect `filename-suggested-map.json` in the build artifacts or in the repo root after running the script locally.

If you want, I'll:

- produce a ready-to-apply patch (diff) that adds the CI YAML and the scripts to your repo, or
- generate a short shell snippet you can run locally to simulate the CI steps on macOS, Linux, or Windows.

on mac, use `brew pre-commit`. `brew` packages always will be on path. on windows - use `winget python`. update docs

## Updated filename policy (ready to commit)

markdown

# Filename Policy

**Goal:** Filenames in the repository must be ASCII-safe, predictable, and cross-platform friendly.

### Requirements

- Transliteration is mandatory. All filenames are transliterated to ASCII using ``Unidecode``.
- Filenames are normalized to NFC before transliteration.
- Whitespace is replaced with ``-``.
- Forbidden characters ``<>:"/\\|?*'`` are replaced with ``_``.
- Control and invisible characters are removed.
- Filenames are lowercased.
- Trailing dots and spaces are removed.
- Windows reserved names `CON PRN AUX NUL COM1..COM9 LPT1..LPT9` are avoided; if encountered, ``_file`` is appended.
- If transliteration causes collisions, numeric suffixes ``-1`` ``-2`` are appended.

### Process

- New commits are validated by a pre-commit hook and will be rejected if filenames violate the policy.
- A one-time migration script ``migrate_filenames.py`` is used to rename existing files and produce ``filename-map.json``
- ``filename-map.json`` maps original filenames to stored filenames for user reference.

### Tools required

- Python 3.8+ (3.10+ recommended)
- ``Unidecode`` Python package (mandatory)
- ``pre-commit`` (for hook installation)

### Notes

- macOS users may set ``git config --global core.precomposeunicode true`` to reduce Git/HFS filename mismatches, but do
- Migration should be performed on Linux when possible; macOS is acceptable if Homebrew Python is used and you follow

## Onboarding checklist for contributors

## Onboarding checklist for contributors

- **macOS (recommended setup using Homebrew)**

1. Install Homebrew if needed:

```
bash
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Install Homebrew Python and pre-commit:

```
bash
brew update
brew install python
brew install pre-commit
```

Homebrew packages are on PATH by default on macOS.

3. Install Unidecode with pip:

```
bash
python3 -m pip install --upgrade --user Unidecode
```

4. From the repository root:

```
bash
pre-commit install
pre-commit run --all-files
```

5. If the hook rejects a commit, inspect `.filename-suggested-map.json` in the repo root, rename files locally or run the migration on Linux, stage, and commit again.

- **Linux**

1. Install Python and pip (example Ubuntu):

```
bash
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
```

2. Install pre-commit and Unidecode:

```
bash
python3 -m pip install --user pre-commit Unidecode
export PATH="$HOME/.local/bin:$PATH"
```

3. From the repository root:

```
bash
pre-commit install
pre-commit run --all-files
```

- **Windows (winget Python)**

1. Install Python via winget:

```
powershell
winget install --id Python.Python.3 -e --silent
```

2. Install pre-commit and Unidecode:

```
powershell
python -m pip install --user pre-commit Unidecode
```

3. Ensure the user Scripts folder is on PATH for the session or permanently. Typical path:

```
Code
%USERPROFILE%\AppData\Roaming\Python\Python3x\Scripts
```

4. From the repository root:

```
powershell
pre-commit install
pre-commit run --all-files
```

## Exact commands to add to docs

- macOS quick install

bash

```
brew update
brew install python
brew install pre-commit
python3 -m pip install --upgrade --user Unidecode
pre-commit install
pre-commit run --all-files
```

- Linux quick install

bash

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
python3 -m pip install --user pre-commit Unidecode
export PATH="$HOME/.local/bin:$PATH"
pre-commit install
pre-commit run --all-files
```

- Windows quick install

powershell

```
winget install --id Python.Python.3 -e --silent
python -m pip install --user pre-commit Unidecode
ensure user Scripts folder is on PATH
pre-commit install
pre-commit run --all-files
```

## Updated Codemagic CI job (complete)

Add this codemagic.yaml workflow to run filename checks on macOS, Linux, and Windows. The macOS job uses Homebrew to install pre-commit as requested; the Windows job uses winget to install Python.

yaml

```
workflows:
 check-filenames-matrix:
 name: Check Filenames Matrix
 max_build_duration: 60
 environment:
 vars:
 PYTHONUSERBASE: $HOME/.local
 scripts:
 - name: Prepare workspace
 script: |
 set -e
 echo "Runner platform: $CM_BUILD_PLATFORM"
 cd "$CM_BUILD_DIR"
 - name: macOS install
 script: |
 if ["$CM_BUILD_PLATFORM" = "macos"]; then
 echo "macOS runner: install pre-commit via brew and Unidecode via pip"
 brew update || true
 brew install pre-commit || true
 python3 -m pip install --upgrade --user Unidecode
 # pre-commit installed by brew is on PATH
 pre-commit --version || true
 fi
 - name: Linux install
 script: |
 if ["$CM_BUILD_PLATFORM" = "linux"]; then
 echo "Linux runner: install pre-commit and Unidecode via pip"
 python3 -m pip install --user --upgrade pip
 python3 -m pip install --user pre-commit Unidecode
 export PATH="$HOME/.local/bin:$PATH"
 pre-commit --version || true
 fi
 - name: Windows install
 script: |
 if ["$CM_BUILD_PLATFORM" = "windows"]; then
 echo "Windows runner: install Python via winget and Python packages"
 winget install --id Python.Python.3 -e --silent || true
 python -m pip install --user pre-commit Unidecode
 # Add user Scripts to PATH for this session
 export PATH="%APPDATA%\Python\Scripts;%PATH%" || true
 pre-commit --version || true
 fi
 - name: Run filename sanitizer
 script: |
 set -e
 cd "$CM_BUILD_DIR"
 echo "Running filename sanitizer on all files..."
```

```

 echo "Running sanitize_commit_filenames.py"
 # Use python3 where available; on Windows python is used
 python3 sanitize_commit_filenames.py || (echo "Filename policy violations detected. See .filename-suggested
- name: Run pre-commit hooks
 script: |
 set -e
 cd "$CM_BUILD_DIR"
 pre-commit run --all-files || (echo "pre-commit hooks failed. Fix issues and push again." && exit 1)
triggers:
- push
- pull_request

```

## Migration and verification reminders

- **Do migration on Linux** when possible. If you must use macOS, use Homebrew Python and follow the migration steps carefully.
- After migration, **test a fresh clone on Windows** to ensure no forbidden names remain.
- Commit filename-map.json and FILENAME\_POLICY.md with the migration commit so everyone has the mapping and policy.

## Docs update summary to commit

- Replace FILENAME\_POLICY.md with the updated version above.
- Add or update README or CONTRIBUTING with the onboarding checklist and exact install commands.
- Add codemagic.yaml with the CI workflow above.
- Ensure sanitize\_filename.py, sanitize\_commit\_filenames.py, migrate\_filenames.py, .pre-commit-config.yaml, and example filename-map.json are present in the repo root.

I've updated the policy, onboarding checklist, exact install commands for Homebrew and winget, and the full Codemagic CI job. If you want, I'll produce a ready patch (diff) that adds or updates these files in your repo, or generate the single-line shell sequence to run the migration dry-run and apply on your Linux machine. Which should I prepare next?

k, i am waiting for linux machine

## Ready steps to run on your Linux machine (when it's available)

Below is a single, safe, copy-pasteable sequence you can run on a fresh Linux environment (Ubuntu/Debian example). It installs required tools, performs a dry run, shows the proposed mapping, and — only when you explicitly run the `--apply` step — performs `git mv` operations and writes `filename-map.json`.

**Important:** run these from the repository root. Do this on a branch and push that branch so you can review and revert if needed.

### 1) Install Python and required packages

bash

```
Ubuntu / Debian example
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git

Install tools into user site (no sudo)
python3 -m pip install --user pre-commit unicodecode

Ensure user bin is on PATH for this session
export PATH="$HOME/.local/bin:$PATH"
```

### 2) Add the scripts and config (if not already in repo)

Create these files in the repo root if you haven't yet:

- `sanitize_filename.py` (provided earlier)
- `sanitize_commit_filenames.py` (pre-commit hook script)
- `migrate_filenames.py` (migration tool)
- `.pre-commit-config.yaml`
- `FILENAME_POLICY.md`

Make them executable where appropriate:

bash

```
git add sanitize_filename.py sanitize_commit_filenames.py migrate_filenames.py .pre-commit-config.yaml FILENAME_POLICY.md
git commit -m "Add filename sanitization scripts, pre-commit config, and policy" || true
```

### 3) Install pre-commit hooks locally

bash

```
pre-commit install
pre-commit run --all-files || true
```

If `pre-commit run --all-files` reports filename violations, the hook will have written `.filename-suggested-map.json` to the repo root. Inspect it before proceeding.

### 4) Dry-run migration (no changes yet)

bash

```
Create a branch for the migration
git checkout -b migrate/filenames

Dry-run: writes filename-map-dryrun.json to repo root
python3 migrate_filenames.py --dry-run
Inspect the file
less filename-map-dryrun.json
```

- **Inspect** `filename-map-dryrun.json` carefully for collisions or undesired transliterations.
- If collisions exist, edit the dryrun JSON to resolve names or plan manual fixes.

### 5) Resolve collisions (if any)

- If `migrate_filenames.py` printed collisions, open `filename-map-dryrun.json` and decide unique targets (or let the

apply step append numeric suffixes).

- Make any manual decisions now; you can also rename files locally and stage them before applying.

## 6) Apply migration (performs `git mv` and writes `filename-map.json`)

bash

```
When ready to apply:
python3 migrate_filenames.py --apply

Review changes
git status
git diff --name-status

Add the mapping and commit
git add filename-map.json
git commit -m "Migrate filenames to ASCII-safe names and add filename-map.json"
git push --set-upstream origin migrate/filenames
```

## 7) Verify on other platforms

- **macOS:** ask macOS devs to `git pull` and ensure they have Homebrew Python and `pre-commit` installed (or brew install `pre-commit`), and run `pre-commit run --all-files`. Recommend `git config --global core.precomposeunicode true` as optional help.
- **Windows:** perform a fresh clone on a Windows machine (or ask a teammate) to ensure checkout succeeds and no forbidden names remain.

## 8) Rollback plan (if something goes wrong)

bash

```
If you need to revert the migration commit:
git checkout main
git revert <commit-hash-of-migration> # creates a revert commit
or reset branch if not pushed:
git reset --hard origin/main
```

Keep the migration branch until everyone has updated and CI passes.

## Quick verification and CI

- After pushing the migration branch, open a PR. Codemagic CI (configured earlier) will run `sanitize_commit_filenames.py` and `pre-commit run --all-files`. Fix any issues the CI reports, update mapping if needed, and re-run the migration apply if you changed decisions.
- Once CI passes and teammates confirm Windows checkout works, merge the migration branch into main.

## Safety checklist before you run `--apply`

- [ ] You are on a dedicated branch (e.g., `migrate/filenames`).
- [ ] You have a recent backup or pushed branch to remote.
- [ ] You inspected `filename-map-dryrun.json` and resolved collisions.
- [ ] You understand that `git mv` will preserve history for renamed files.
- [ ] You will test a fresh clone on Windows after merging.

## If you want now

I can prepare either (pick one) and deliver it immediately so you can run it on Linux when ready:

- **A. Patch (diff)** that adds all scripts, `.pre-commit-config.yaml`, `FILENAME_POLICY.md`, and example `filename-map.json` to the repo root.
- **B. One-line shell snippet** that runs the exact sequence above (installs deps, dry-run, apply) for Ubuntu.

Tell me which you want and I'll produce it now so it's waiting for your Linux machine.

k. file name is "Gmail - You're eligible for a Reward - HDFC Bank MoneyBack+ Credit Card.pdf". what will be the mapped file name

#### Code

```
gmail-you're-eligible-for-a-reward-hdfc-bank-moneyback+-credit-card.pdf
```

### Gmail - You're eligible for a Reward - HDFC Bank MoneyBack+ Credit Card

Applying the transliteration + sanitization rules we defined:

1. **Normalize Unicode** → no special characters here, just ASCII.
2. **Remove control/invisible characters** → none present.
3. **Transliterate** → not needed (already English).
4. **Whitespace** → - → spaces become dashes.
5. **Forbidden characters** → none (<>.:'/\|?\*)
6. **Lowercase** → everything lowercase.
7. **Trim trailing dots/spaces** → none.
8. **Reserved names** → not matching Windows reserved names.

Resulting mapped filename:

#### Code

```
gmail-you're-eligible-for-a-reward-hdfc-bank-moneyback+-credit-card
```

If you save it as a file, you'd append the extension (e.g., .pdf):

#### Code

```
gmail-you're-eligible-for-a-reward-hdfc-bank-moneyback+-credit-card.pdf
```

💡 **Note:** The + character is technically allowed on all major platforms, but if you want to be ultra-safe (especially for scripts or URLs), you could replace + with plus:

#### Code

```
gmail-youre-eligible-for-a-reward-hdfc-bank-moneybackplus-credit-card.pdf
```

Would you prefer to **keep symbols like +** in mapped names, or should we normalize them into words like `plus` for maximum portability?

We have to need url safe and script safe to

## Filename strategy comparison

| Approach                                    | URL safe | Shell/script safe          | Human readable     |
|---------------------------------------------|----------|----------------------------|--------------------|
| Percent-encode reserved chars               | Yes      | Yes                        | Low (hard to read) |
| Replace symbols with words (e.g., + → plus) | Yes      | Yes                        | High               |
| Replace symbols with - or _                 | Yes      | Mostly (some symbols lost) | Medium             |

## Recommendation

**Use transliteration → symbol-to-word mapping → strict character whitelist. Why:** produces filenames that are RFC3986-friendly, safe in shells and scripts, easy to read, and deterministic for mapping and search.

## Policy summary

- **Transliterate** all Unicode to ASCII using `unicodedata`.
- **Map common symbols** to words: + → plus, & → and, @ → at, % → pct, # → hash.
- **Remove** apostrophes and other quotes.
- **Allow only** lowercase letters a–z, digits 0–9, hyphen -, underscore \_, and dot . (for extension).
- **Replace any other character** with -. Collapse repeated - into one. Trim leading/trailing - or ..
- **Ensure** filename does not start with - or .; if it would, prefix with file-.
- **Avoid Windows reserved names** by appending \_file if needed.
- **Limit length** to a safe value (recommend 200 characters including extension).

## Python implementation (drop into `sanitize_filename.py`)

python

```
#!/usr/bin/env python3
import unicodedata, re, os, sys
from unicode import unicode

SYMBOL_MAP = {
 '+': 'plus', '&': 'and', '@': 'at', '%': 'pct', '#': 'hash', '/': '-', '\\': '-'
}

WINDOWS_RESERVED = {"CON", "PRN", "AUX", "NUL"} | {f"COM{i}" for i in range(1,10)} | {f"LPT{i}" for i in range(1,10)}

def sanitize_basename(name):
 name = unicodedata.normalize('NFC', name)
 name = ''.join(ch for ch in name if unicodedata.category(ch)[0] != 'C')
 name = unicode(name)
 # map symbols to words first
 for s, w in SYMBOL_MAP.items():
 name = name.replace(s, f"-{w}-")
 # remove quotes/apostrophes
 name = name.replace("'", "").replace('"', '')
 # replace any non-whitelisted char with hyphen
 name = re.sub(r'[^A-Za-z0-9._-]+', '-', name)
 # collapse multiple hyphens/underscores
 name = re.sub(r'[_-]{2,}', lambda m: m.group(0)[0], name)
 name = name.strip('._-').lower()
 if not name:
 name = 'file'
 if name.upper() in WINDOWS_RESERVED:
 name = name + '_file'
 # avoid leading dash or dot
 if name[0] in '-.':
 name = 'file-' + name.lstrip('-.')
 # enforce max length (200 chars)
 if len(name) > 200:
 base, ext = os.path.splitext(name)
 name = base[:200 - len(ext)] + ext
 return name
```

## Example mapping

Original: Gmail - You're eligible for a Reward - HDFC Bank MoneyBack+ Credit Card.pdf

Step results:

Step results.

- Transliterate: same (ASCII)
- Symbol map: + → plus
- Remove apostrophe: You're
- Spaces → -
- Lowercase

#### Final mapped filename

Code

```
gmail-youre-eligible-for-a-reward-hdfc-bank-moneyback-plus-credit-card.pdf
```

#### Integration notes

- **Update** `sanitize_filename.py` in repo with the code above.
- **Pre-commit hook:** keep `sanitize_commit_filenames.py` to call `sanitize_basename` and reject commits when staged names differ. Write suggested mapping to `root/.filename-suggested-map.json`.
- **Migration:** run `migrate_filenames.py --dry-run`, inspect `filename-map-dryrun.json`, resolve collisions, then `--apply`.
- **CI:** run the same sanitizer in CI magic to block PRs with unsafe names.
- **Documentation:** update `FILENAME_POLICY.md` to state symbol→word mappings and the whitelist.

If you want, I'll update the repo scripts now to apply this exact mapping logic and show the dry-run result for your repository example filenames.

Gmail - ProPeers App New DSA Patterns Edition at Lowest Price.pdf

Applying the **URL-safe + script-safe policy** we defined:

1. **Normalize Unicode** → already ASCII.
2. **Remove control/invisible characters** → none present.
3. **Transliterate** → not needed (English text).
4. **Symbol mapping** → no +, &, @, etc. here.
5. **Remove apostrophes/quotes** → none.
6. **Whitespace** → - → spaces become dashes.
7. **Lowercase** → all letters lowercase.
8. **Whitelist only** [a-z0-9.\_-].
9. **Trim leading/trailing dots/spaces/hyphens**.
10. **Reserved names check** → not matching Windows reserved names.

#### Final mapped filename:

Code

```
gmail-propeers-app-new-dsa-patterns-edition-at-lowest-price.pdf
```

This is safe for:

- **URLs** (no unsafe symbols, only lowercase letters, digits, hyphens, and dot for extension).
- **Scripts/shells** (no spaces, quotes, or special characters).

👉 If you'd like, I can show you how the sanitizer script processes this step-by-step so you can see exactly how each rule applies. Would you like me to demonstrate that?

DEMO WATERMARK